

# Pthread/OpenMP 多线程编程作业

2411567 陈涵

2026.05.26

## 目录

|          |                                 |           |
|----------|---------------------------------|-----------|
| <b>1</b> | <b>问题描述</b>                     | <b>2</b>  |
| 1.1      | 期末研究报告宏观问题 . . . . .            | 2         |
| 1.2      | 本次多线程编程实验子问题 . . . . .          | 2         |
| <b>2</b> | <b>多线程算法设计与实现</b>               | <b>2</b>  |
| 2.1      | PCFG 口令猜测流程 . . . . .           | 2         |
| 2.2      | OpenMP 并行实现 . . . . .           | 3         |
| 2.3      | Pthread 并行实现 . . . . .          | 3         |
| 2.4      | 实验中的优化 . . . . .                | 3         |
| 2.5      | 算法代码实现 . . . . .                | 4         |
| <b>3</b> | <b>实验及结果分析</b>                  | <b>8</b>  |
| 3.1      | 实验结果 . . . . .                  | 8         |
| 3.2      | 结果分析 . . . . .                  | 8         |
| 3.2.1    | OpenMP 与 Pthread 性能比较 . . . . . | 8         |
| 3.2.2    | 队列优化与进阶要求实现 . . . . .           | 9         |
| 3.2.3    | 大规模数据下的性能变化 . . . . .           | 10        |
| 3.2.4    | 任务粒度分析与阈值控制 . . . . .           | 11        |
| <b>4</b> | <b>总结</b>                       | <b>14</b> |

## 1 问题描述

### 1.1 期末研究报告宏观问题

本课题旨在构建一套高效的混合并行口令猜测系统,需融合 MPI(跨节点任务分发)、OpenMP/Pthread 与 SIMD/GPU 等技术。核心研究问题在于:如何突破优先队列操作的锁竞争瓶颈、实现细粒度负载均衡、缓解 CPU-GPU/内存数据传输延迟,并量化分析“口令生成”与“哈希计算”在完整链路中的算力占比,从而设计最优的异构并行流水线。

### 1.2 本次多线程编程实验子问题

本次实验主要针对 PCFG 口令猜测系统中的猜测生成 (Guess Generation) 阶段进行并行化优化,分别使用 OpenMP 与 Pthread 两种多线程编程方式实现并行猜测生成并进行性能对比。

实验重点包括:

- 分析 Generate 函数中的可并行部分;
- 使用 OpenMP 实现 for 循环级并行;
- 使用 Pthread 实现手动线程划分;
- 比较串行、OpenMP 与 Pthread 版本的性能差异;
- 分析线程开销、任务粒度与性能之间的关系。

## 2 多线程算法设计与实现

### 2.1 PCFG 口令猜测流程

本实验使用基于概率上下文无关文法 (PCFG) 的口令猜测模型,代码均存于此仓库的 lab3 中。整体流程包括:

1. 对训练集口令进行 segment 划分;
2. 统计 Pre-terminal (PT) 与 segment 频率;
3. 使用优先队列维护当前概率最高的 PT;
4. 调用 Generate 函数展开最后一个 segment;
5. 生成 guess 后进行 MD5 哈希验证。

其中,Generate 函数会对最后一个 segment 的所有候选 value 进行遍历生成,不同 segment 之间没有数据依赖,不同 candidate 之间相互独立,因此天然具有较好的数据并行特性。

## 2.2 OpenMP 并行实现

OpenMP 版本主要使用：

```
1 #pragma omp parallel for schedule(static)
```

对 Generate 函数中的 for 循环进行并行化。

为了减少线程之间的竞争, 每个线程使用局部 vector 缓存生成结果, 最终再统一合并到全局 guesses 中。

## 2.3 Pthread 并行实现

Pthread 版本采用手动线程划分方式。主线程根据待生成 guess 数量划分任务区间, 每个线程负责一个 chunk 的数据生成。

线程参数结构如下：

```
1 struct ThreadArgs
2 {
3     int start_idx;
4     int end_idx;
5     string prefix;
6     segment *seg_ptr;
7     vector<string> local_result;
8 };
```

线程内部使用局部 vector 保存生成结果, 避免多线程同时写入共享 vector 导致锁竞争。

线程结束后, 由主线程统一 merge 结果。

## 2.4 实验中的优化

为了减少额外开销, 实验中进行了如下优化：

- 使用 thread-local vector 减少竞争；
- 使用 reserve 减少 vector 扩容；
- 使用 move 语义减少 string 拷贝；
- 使用 static schedule 减少 OpenMP 调度开销。

由于多个线程同时写入全局 guesses vector 可能产生访存冲突, 因此实验中采用 thread-local vector 方式, 每个线程先写入局部缓存, 最终再由主线程统一 merge, 从而避免频繁加锁带来的临界区开销。并且, 实验中采用均匀 chunk 划分方式, 尽量保证各线程处理的 candidate 数量接近, 从而降低线程间负载不均衡问题。

在早期的实现中，最终发现对于原本 10000000 的数据规模，基础的细粒度的并行优化效果并不明显，甚至出现了负优化的情况，OpenMP 与 Pthread 版本的代码均比串行版本代码慢了零点几秒。

此外，为了减少共享节点带来的性能波动，保证每次测试结果数据的可靠性，实验运行时使用以下命令提交测试以减少共享服务器环境中的资源竞争，使得实验结果更加稳定：

---

```
1 bash test.sh 2 4
```

---

## 2.5 算法代码实现

以下为 guessing.cpp 中 Generate() 函数的相关代码，已添加宏控制，尚未加入进阶要求中的优化：

---

```
1 // 这个函数是 PCFG 并行化算法的主要载体
2 // 尽量看懂，然后进行并行实现
3 void PriorityQueue::Generate(PT pt)
4 {
5     // 计算 PT 的概率，这里主要是给 PT 的概率进行初始化
6     CalProb(pt);
7
8     // 对于只有一个 segment 的 PT，直接遍历生成其中的所有 value 即可
9     if (pt.content.size() == 1)
10     {
11         // 指向最后一个 segment 的指针，这个指针实际指向模型中的统计数据
12         segment *a;
13         // 在模型中定位到这个 segment
14         if (pt.content[0].type == 1)
15         {
16             a = &m.letters[m.FindLetter(pt.content[0])];
17         }
18         if (pt.content[0].type == 2)
19         {
20             a = &m.digits[m.FindDigit(pt.content[0])];
21         }
22         if (pt.content[0].type == 3)
23         {
24             a = &m.symbols[m.FindSymbol(pt.content[0])];
25         }
26
27         // Multi-thread TODO:
28         // 这个 for 循环就是你需要进行并行化的主要部分了，特别是在多线程 & GPU 编程任务中
29         // 可以看到，这个循环本质上就是把模型中一个 segment 的所有 value，赋值到 PT 中，形成一系列新的猜测
30         // 这个过程是可以高度并行化的
31         #ifdef SERIAL
32         for (int i = 0; i < pt.max_indices[0]; i += 1)
33         {
34             string guess = a->ordered_values[i];
35             // cout << guess << endl;
36             guesses.emplace_back(guess);
37             total_guesses += 1;
38         }
39         #endif
```

---

```

40
41     #ifdef OPENMP
42     vector<vector<string>> thread_buf(4);
43     #pragma omp parallel num_threads(4)
44     {
45         int tid = omp_get_thread_num();
46         int total = pt.max_indices[0];
47         thread_buf[tid].reserve((total + 3) / 4);
48
49         #pragma omp for schedule(static)
50         for (int i = 0; i < total; i += 1) {
51             string guess = a->ordered_values[i];
52             thread_buf[tid].emplace_back(guess);
53         }
54     }
55     guesses.reserve(guesses.size() + pt.max_indices[0]);
56     for (int t = 0; t < 4; t += 1) {
57         for (string &guess : thread_buf[t]) {
58             guesses.emplace_back(move(guess));
59             total_guesses += 1;
60         }
61     }
62     #endif
63
64     #ifdef PTHREAD
65     pthread_t threads[NUM_THREADS];
66     ThreadArgs args[NUM_THREADS];
67
68     int total = pt.max_indices[0];
69     int chunk = (total + NUM_THREADS - 1) / NUM_THREADS;
70     for (int t = 0; t < NUM_THREADS; t += 1)
71     {
72         args[t].start_idx = t * chunk;
73         args[t].end_idx = min((t + 1) * chunk, total);
74         args[t].prefix = "";
75         args[t].seg_ptr = a;
76     }
77     for (int t = 0; t < NUM_THREADS; t += 1)
78     {
79         pthread_create(&threads[t], NULL, thread_generate, &args[t]);
80     }
81     for (int t = 0; t < NUM_THREADS; t += 1)
82     {
83         pthread_join(threads[t], NULL);
84     }
85     for (int t = 0; t < NUM_THREADS; t += 1)
86     {
87         for (string &guess : args[t].local_result)
88         {
89             guesses.emplace_back(move(guess));
90             total_guesses += 1;
91         }
92     }
93     #endif
94 }

```

```

95     else
96     {
97         string guess;
98         int seg_idx = 0;
99         // 这个 for 循环的作用：给当前 PT 的所有 segment 赋予实际的值（最后一个 segment 除外）
100        // segment 值根据 curr_indices 中对应的值加以确定
101        // 这个 for 循环你看不懂也没太大问题，并行算法不涉及这里的加速
102        for (int idx : pt.curr_indices)
103        {
104            if (pt.content[seg_idx].type == 1)
105            {
106                guess += m.letters[m.FindLetter(pt.content[seg_idx])].ordered_values[idx];
107            }
108            if (pt.content[seg_idx].type == 2)
109            {
110                guess += m.digits[m.FindDigit(pt.content[seg_idx])].ordered_values[idx];
111            }
112            if (pt.content[seg_idx].type == 3)
113            {
114                guess += m.symbols[m.FindSymbol(pt.content[seg_idx])].ordered_values[idx];
115            }
116            seg_idx += 1;
117            if (seg_idx == pt.content.size() - 1)
118            {
119                break;
120            }
121        }
122
123        // 指向最后一个 segment 的指针，这个指针实际指向模型中的统计数据
124        segment *a;
125        if (pt.content[pt.content.size() - 1].type == 1)
126        {
127            a = &m.letters[m.FindLetter(pt.content[pt.content.size() - 1])];
128        }
129        if (pt.content[pt.content.size() - 1].type == 2)
130        {
131            a = &m.digits[m.FindDigit(pt.content[pt.content.size() - 1])];
132        }
133        if (pt.content[pt.content.size() - 1].type == 3)
134        {
135            a = &m.symbols[m.FindSymbol(pt.content[pt.content.size() - 1])];
136        }
137
138        // Multi-thread TODO:
139        // 这个 for 循环就是你需要进行并行化的主要部分了，特别是在多线程 & GPU 编程任务中
140        // 可以看到，这个循环本质上就是把模型中一个 segment 的所有 value，赋值到 PT 中，形成一系列新的猜测
141        // 这个过程是可以高度并行化的
142        #ifdef SERIAL
143        for (int i = 0; i < pt.max_indices[pt.content.size() - 1]; i += 1)
144        {
145            string temp = guess + a->ordered_values[i];
146            // cout << temp << endl;
147            guesses.emplace_back(temp);
148            total_guesses += 1;
149        }

```

```
150     #endif
151
152     #ifdef OPENMP
153     vector<vector<string>> thread_buf(4);
154     #pragma omp parallel num_threads(4)
155     {
156         int tid = omp_get_thread_num();
157         int total = pt.max_indices[pt.content.size() - 1];
158         thread_buf[tid].reserve((total + 3) / 4);
159
160         #pragma omp for schedule(static)
161         for (int i = 0; i < total; i += 1) {
162             string temp = guess + a->ordered_values[i];
163             thread_buf[tid].emplace_back(temp);
164         }
165     }
166     guesses.reserve(guesses.size() + pt.max_indices[pt.content.size() - 1]);
167     for (int t = 0; t < 4; t += 1) {
168         for (string &temp : thread_buf[t]) {
169             guesses.emplace_back(move(temp));
170             total_guesses += 1;
171         }
172     }
173     #endif
174
175     #ifdef PTHREAD
176     pthread_t threads[NUM_THREADS];
177     ThreadArgs args[NUM_THREADS];
178
179     int total = pt.max_indices[pt.content.size() - 1];
180     int chunk = (total + NUM_THREADS - 1) / NUM_THREADS;
181     for (int t = 0; t < NUM_THREADS; t += 1)
182     {
183         args[t].start_idx = t * chunk;
184         args[t].end_idx = min((t + 1) * chunk, total);
185         args[t].prefix = guess;
186         args[t].seg_ptr = a;
187     }
188     for (int t = 0; t < NUM_THREADS; t += 1)
189     {
190         pthread_create(&threads[t], NULL, thread_generate, &args[t]);
191     }
192     for (int t = 0; t < NUM_THREADS; t += 1)
193     {
194         pthread_join(threads[t], NULL);
195     }
196     for (int t = 0; t < NUM_THREADS; t += 1)
197     {
198         for (string &temp : args[t].local_result)
199         {
200             guesses.emplace_back(move(temp));
201             total_guesses += 1;
202         }
203     }
204     #endif
```

```

205 }
206 }

```

### 3 实验及结果分析

#### 3.1 实验结果

| Version | Guess Time (s) | Hash Time (s) | Train Time (s) |
|---------|----------------|---------------|----------------|
| Serial  | 0.580200       | 5.44876       | 28.1047        |
| OpenMP  | 0.882933       | 5.55631       | 26.2397        |
| Pthread | 0.643628       | 5.44085       | 25.8576        |

表 1: 不同版本性能对比

为了方便管理各个版本的猜测代码，本次实验在 PCFG.h 头部添加了宏控制，测试时可通过手动对相应的宏取消注释以选择对应版本的代码。

此外，为了保证并行化方法不改变原有代码逻辑，确保口令猜测结果的正确性，本次实验将群里发的 correctness.cpp 中与正确性验证有关的内容加入到了原有的 main.cpp 中，且在开头添加了宏以控制开关。经过多次测试，攻破率始终一致，说明代码逻辑正确。具体值如下：

Cracked: 358217

Crack Rate: 35.8217%

#### 3.2 结果分析

##### 3.2.1 OpenMP 与 Pthread 性能比较

基础版本实验结果如下：

| Version | Guess Time (s) | Hash Time (s) | Train Time (s) |
|---------|----------------|---------------|----------------|
| Serial  | 0.580200       | 5.44876       | 28.1047        |
| OpenMP  | 0.882933       | 5.55631       | 26.2397        |
| Pthread | 0.643628       | 5.44085       | 25.8576        |

表 2: 基础版本性能测试

可以发现，在原始实现中，并行版本并未获得加速效果，甚至出现了负优化。

其中：

- OpenMP 版本相比串行版本慢约 52%；
- Pthread 版本相比串行版本慢约 11%。

分析代码后发现，虽然 Generate 函数中的 for 循环不存在数据依赖，具有天然的数据并行性，但单次 Generate 任务规模通常较小，而线程创建、调度、同步以及结果 merge 本身存在额外开销。



此外：

- OpenMP 运行时需要维护线程池与调度器；
- Pthread 需要频繁进行 `pthread_create` 与 `pthread_join`；
- 多线程版本最终仍需 merge 局部 vector。

因此，在数据规模较小时，并行带来的收益无法覆盖额外线程管理成本。

同时，从结果可以看出，Pthread 版本整体略优于 OpenMP 版本。这是因为本实验中的 Pthread 采用了较为直接的手动 chunk 划分方式，调度逻辑更简单，因此运行时开销略低。

### 3.2.2 队列优化与进阶要求实现

根据实验指导书中的进阶要求，进一步分析程序运行过程后发现：

随着 Generate 部分逐渐优化，性能瓶颈开始从字符串生成转移到 priority queue 维护过程。

原始实现中的队列插入复杂度较高，因此本实验进一步采用 STL heap 结构优化队列维护，将原本的 PopNext 函数注释到更换为堆版本，由于算法复杂度下降，较为有效地解决了瓶颈：

```

1 // 堆版本 PopNext
2 void PriorityQueue::PopNext()
3 {
4     PT top_pt = priority.front();
5
6     pop_heap(priority.begin(), priority.end(), PTCompare());
7     priority.pop_back();
8
9     Generate(top_pt);
10
11     vector<PT> new_pts = top_pt.NewPTs();
12
13     for (PT pt : new_pts)
14     {
15         CalProb(pt);
16         priority.emplace_back(pt);
17         push_heap(priority.begin(), priority.end(), PTCompare());
18     }
19 }
20

```

优化后实验结果如下：

| Version | Guess Time (s) | Crack Rate |
|---------|----------------|------------|
| Serial  | 0.423912       | 38.2853%   |
| OpenMP  | 0.669964       | 38.2853%   |
| Pthread | 1.15175        | 38.2853%   |

表 3: 队列优化后性能测试

可以看到：

- 串行版本 Guess 阶段进一步加速；
- OpenMP 也有加速效果但仍存在一定线程开销；
- Pthread 版本反而进一步变慢。

分析原因后认为：

队列优化后，Generate 阶段本身耗时下降较明显，此时线程创建与同步开销在总时间中的占比进一步提高，因此 Pthread 中的频繁线程创建问题被进一步放大。

与此同时，Crack Rate 由 35.8217% 提升至 38.2853%，说明新的队列维护方式能够更高效地维持高概率 PT 的扩展顺序，从而提升整体猜测效果。

### 3.2.3 大规模数据下的性能变化

经过查询，得知可能是数据集规模太小导致并行版本体现不出优化效果，随后尝试将生成规模扩大 10 倍，从 10000000 提高到 100000000，实验结果如下：

| Version | Guess Time (s) | Hash Time (s) |
|---------|----------------|---------------|
| Serial  | 4.01367        | 57.7651       |
| OpenMP  | 4.72775        | 57.5881       |
| Pthread | 7.3796         | 53.9731       |

表 4: 10 倍数据规模性能测试

```
Guess time:4.1288seconds
SIMD Hash time:59.4172seconds
Train time:23.7929seconds
Cracked:444992
Crack Rate: 44.4992%
```

图 1: Serial (10 倍数据量)

此时可以明显观察到：

- Hash 阶段已经远远超过 Guess 阶段；
- SIMD MD5 逐渐成为系统主要瓶颈；
- Generate 阶段优化对整体程序影响开始减弱；
- OpenMP 与串行版本的差距进一步缩小。

同时,大规模数据下 Pthread 性能下降更加明显,效率更低,经过分析后发现由于当前实现中每次 Generate 都会重新创建线程,因此当 Generate 调用次数大幅增加时,pthread\_create 与 pthread\_join 累计开销迅速增大。

相比之下,OpenMP 内部维护线程池,因此整体开销相对更稳定。

### 3.2.4 任务粒度分析与阈值控制

为了对 Pthread 版本的代码进行更深入的优化,进一步分析 Generate 函数后发现,并不是所有 Generate 任务都具有足够大的任务规模。

很多 segment 对应的 candidate 数量较少,如果仍然强行进行多线程并行化,则:

- 线程创建时间甚至超过实际计算时间;
- merge 局部 vector 也会产生额外成本;
- OpenMP 调度开销无法有效摊销。

因此,本实验进一步加入了任务粒度阈值控制,通过以下的宏控制并行阈值:

---

```
1 #define PARALLEL_THRESHOLD 100
```

---

当任务规模小于阈值时,直接退化为串行执行,仅在任务规模足够大时启动并行化。经过多次测试,阈值取 100 会有较好的优化效果。

加入阈值控制之后的 Pthread 相关代码如下:

---

```
1 // 小于阈值时直接采用串行版本代码
2 #ifdef PTHREAD
3 int total = pt.max_indices[0];
4
5 if (total < PARALLEL_THRESHOLD)
6 {
7     for (int i = 0; i < total; i += 1)
8     {
9         string guess = a->ordered_values[i];
10        guesses.emplace_back(guess);
11        total_guesses += 1;
12    }
13 }
14 else
15 {
16     pthread_t threads[NUM_THREADS];
17     ThreadArgs args[NUM_THREADS];
18
19     int chunk = (total + NUM_THREADS - 1) / NUM_THREADS;
20
21     for (int t = 0; t < NUM_THREADS; t += 1)
22     {
23         args[t].start_idx = t * chunk;
```

---

```
24     args[t].end_idx = min((t + 1) * chunk, total);
25     args[t].prefix = "";
26     args[t].seg_ptr = a;
27 }
28
29 for (int t = 0; t < NUM_THREADS; t += 1)
30 {
31     pthread_create(&threads[t], NULL, thread_generate, &args[t]);
32 }
33
34 for (int t = 0; t < NUM_THREADS; t += 1)
35 {
36     pthread_join(threads[t], NULL);
37 }
38
39 for (int t = 0; t < NUM_THREADS; t += 1)
40 {
41     for (string &guess : args[t].local_result)
42     {
43         guesses.emplace_back(move(guess));
44         total_guesses += 1;
45     }
46 }
47 }
48 #endif
49 ...
50 // 对于多 segment 也类似
51 #ifdef PTHREAD
52 int total = pt.max_indices[pt.content.size() - 1];
53
54 if (total < PARALLEL_THRESHOLD)
55 {
56     for (int i = 0; i < total; i += 1)
57     {
58         string temp = guess + a->ordered_values[i];
59         guesses.emplace_back(temp);
60         total_guesses += 1;
61     }
62 }
63 else
64 {
65     pthread_t threads[NUM_THREADS];
66     ThreadArgs args[NUM_THREADS];
67
68     int chunk = (total + NUM_THREADS - 1) / NUM_THREADS;
69
70     for (int t = 0; t < NUM_THREADS; t += 1)
71     {
72         args[t].start_idx = t * chunk;
73         args[t].end_idx = min((t + 1) * chunk, total);
74         args[t].prefix = guess;
75         args[t].seg_ptr = a;
76     }
77
78     for (int t = 0; t < NUM_THREADS; t += 1)
```

```

79 {
80     pthread_create(&threads[t], NULL, thread_generate, &args[t]);
81 }
82
83 for (int t = 0; t < NUM_THREADS; t += 1)
84 {
85     pthread_join(threads[t], NULL);
86 }
87
88 for (int t = 0; t < NUM_THREADS; t += 1)
89 {
90     for (string &temp : args[t].local_result)
91     {
92         guesses.emplace_back(move(temp));
93         total_guesses += 1;
94     }
95 }
96 }
97 #endif

```

Pthread 阈值优化后得到数据结果如下：

| Version             | Guess Time (s) |
|---------------------|----------------|
| Pthread + Threshold | 5.30162        |

表 5: Pthread 阈值控制优化

```

Pthread Guess time:5.95827seconds
SIMD Hash time:57.4351seconds
Train time:23.3214seconds
Cracked:444992
Crack Rate: 44.4992%

```

图 2: Pthread（有阈值控制，效率随服务器性能波动）

OpenMP 阈值优化后得到数据结果如下：

| Version            | Guess Time (s) |
|--------------------|----------------|
| OpenMP + Threshold | 5.68488        |

表 6: OpenMP 阈值控制优化

可以发现：

- Pthread Guess 时间由 7.38s 下降至 5.30s；
- OpenMP Guess 相对于原始版本时间改善效果不佳；

```
OpenMP Guess time:4.97077seconds
SIMD Hash time:58.5567seconds
Train time:23.5194seconds
Cracked:444992
Crack Rate: 44.4992%
```

图 3: OpenMP (无阈值控制)

- 阈值控制有效减少了细粒度任务中的线程管理开销。

因此最终选择保留对 Pthread 的阈值控制，OpenMP 保留原本代码。本次实验说明并行化并非一定带来加速，只有当任务粒度足够大时，并行化才能有效摊销线程创建与调度成本。

## 4 总结

本实验完成了基于 OpenMP 与 Pthread 的 PCFG 口令猜测 Generate 阶段并行化实现，并完成了实验指导书中的队列优化进阶要求。

实验中首先实现了 Generate 阶段的 for 循环并行化，并通过：

- thread-local vector；
- reserve 预分配；
- move 语义；
- static schedule；

降低线程竞争与内存管理开销。

随后进一步分析性能瓶颈，发现：

- 初始阶段主要瓶颈来自线程创建与同步；
- 队列维护优化后，Generate 阶段进一步加速；

实验进一步通过：

- STL heap 优化 priority queue；
- 大规模数据测试；
- 阈值控制优化；
- OpenMP 与 Pthread 对比；

验证了“并行化存在额外代价”这一核心思想。

最终实验表明：

- 多线程并不一定带来加速；
- 任务粒度对并行性能影响极大；
- 合理的阈值控制能够有效减少负优化；

通过本次实验，对线程调度、任务划分、负载均衡以及并行性能分析有了更加深入的理解，也进一步体会到算法结构与体系结构协同优化的重要性。